Saiyed Shizain | CU24250233 | Roll: 48
BTech CSE (A) | 3rd Yr - 5th Sem | 2024-2028

Full-Stack Web Development — BTCS303P

LAB SHEET — 9: CampusConnect Core: Secure REST API (Tasks 1, 4–6)

Scope split with Lab Sheet 10 (documented assumption)

The Lab 9 question file specifies the whole CampusConnect system while the Lab 10 reference implements it fully. This submission covers the secure API core: JWT auth + RBAC, events/announcements CRUD, zod validation, rate limiting, helmet/CORS, Jest tests and a minimal login/events frontend. Lab 10 adds Socket.io realtime, Redis caching, Redux state, benchmarks and Docker.

Task 1 — Authentication & RBAC (bcrypt, JWT, refresh rotation)

Aim. Register/login with bcrypt hashes (10 rounds), 15-min access tokens plus httpOnly 7-day refresh cookies, and role-gated routes.

Theory. Short-lived signed access tokens limit breach windows; refreshTokenVersion revokes sessions; authorize() maps roles to 403s.

```
const mongoose = require("mongoose");

const UserSchema = new mongoose.Schema(
  {
    name: { type: String, required: true, trim: true },
    email: { type: String, required: true, unique: true, lowercase: true, trim: true },
    passwordHash: { type: String, required: true },
    role: { type: String, enum: ["ADMIN", "STUDENT"], default: "STUDENT" },
    refreshTokenVersion: { type: Number, default: 0 },
  },
  { timestamps: true }
);

module.exports = mongoose.models.User || mongoose.model("User", UserSchema);
```

```
const jwt = require("jsonwebtoken");

const ACCESS_SECRET = process.env.JWT_ACCESS_SECRET || "dev_access_secret_change_me";
const REFRESH_SECRET = process.env.JWT_REFRESH_SECRET || "dev_refresh_secret_change_me";

function signAccessToken(user) {
  return jwt.sign(
    { sub: user._id.toString(), role: user.role, email: user.email },
    ACCESS_SECRET,
    { expiresIn: "15m" }
  );
}

function signRefreshToken(user) {
  return jwt.sign(
    { sub: user._id.toString(), v: user.refreshTokenVersion || 0 },
    REFRESH_SECRET,
    { expiresIn: "7d" }
  );
}

function verifyAccessToken(token) {
  return jwt.verify(token, ACCESS_SECRET);
}
```

```
function verifyRefreshToken(token) {
  return jwt.verify(token, REFRESH_SECRET);
}

module.exports = {
  signAccessToken,
  signRefreshToken,
  verifyAccessToken,
  verifyRefreshToken,
};
```

```
const { verifyAccessToken } = require("../utils/jwt");

// Attaches req.user = { sub, role, email } or rejects with 401.
function authenticate(req, res, next) {
  const header = req.headers.authorization || "";
  const token = header.startsWith("Bearer ") ? header.slice(7) : null;
  if (!token) {
    return res.status(401).json({ error: "Missing access token" });
  }
  try {
    req.user = verifyAccessToken(token);
    return next();
  } catch (err) {
    return res.status(401).json({ error: "Invalid or expired access token" });
  }
}

// Usage: authorize("ADMIN") — rejects other roles with 403.
function authorize(...allowedRoles) {
  return (req, res, next) => {
    if (!req.user) {
      return res.status(401).json({ error: "Not authenticated" });
    }
    if (!allowedRoles.includes(req.user.role)) {
      return res.status(403).json({ error: "Forbidden: insufficient role" });
    }
    return next();
  };
}

module.exports = { authenticate, authorize };
```

```
const bcrypt = require("bcryptjs");
const User = require("../models/User");
const { signAccessToken, signRefreshToken, verifyRefreshToken } = require("../utils/jwt"
);

const SALT_ROUNDS = 10;

const REFRESH_COOKIE_OPTS = {
  httpOnly: true,
  secure: process.env.NODE_ENV === "production",
  sameSite: "strict",
  maxAge: 7 * 24 * 60 * 60 * 1000,
  path: "/api/auth",
};

async function register(req, res) {
  const { name, email, password, role } = req.body;
  const existing = await User.findOne({ email });
  if (existing) {
    return res.status(409).json({ error: "Email already registered" });
  }
  const passwordHash = await bcrypt.hash(password, SALT_ROUNDS);
```

```
  const user = await User.create({
    name,
    email,
    passwordHash,
    role: role === "ADMIN" ? "ADMIN" : "STUDENT",
  });
  const accessToken = signAccessToken(user);
  const refreshToken = signRefreshToken(user);
  res.cookie("refreshToken", refreshToken, REFRESH_COOKIE_OPTS);
  return res.status(201).json({
    user: { id: user._id, name: user.name, email: user.email, role: user.role },
    accessToken,
  });
}

async function login(req, res) {
  const { email, password } = req.body;
  const user = await User.findOne({ email });
  if (!user) {
    return res.status(401).json({ error: "Invalid email or password" });
  }
  const valid = await bcrypt.compare(password, user.passwordHash);
  if (!valid) {
    return res.status(401).json({ error: "Invalid email or password" });
  }
  const accessToken = signAccessToken(user);
  const refreshToken = signRefreshToken(user);
  res.cookie("refreshToken", refreshToken, REFRESH_COOKIE_OPTS);
  return res.status(200).json({
    user: { id: user._id, name: user.name, email: user.email, role: user.role },
    accessToken,
  });
}

async function refresh(req, res) {
  const token = req.cookies && req.cookies.refreshToken;
  if (!token) {
    return res.status(401).json({ error: "Missing refresh token" });
  }
... [20 lines truncated for print]
```

```
const express = require("express");
const { z } = require("zod");
const validate = require("../middleware/validate");
const { loginLimiter } = require("../middleware/rateLimiter");
const { register, login, refresh, logout } = require("../controllers/authController");

const router = express.Router();

const registerSchema = z.object({
  name: z.string().min(2).max(100),
  email: z.string().email(),
  password: z.string().min(8, "Password must be at least 8 characters"),
  role: z.enum(["ADMIN", "STUDENT"]).optional(),
});

const loginSchema = z.object({
  email: z.string().email(),
  password: z.string().min(1, "Password is required"),
});

router.post("/register", validate({ body: registerSchema }), register);
router.post("/login", loginLimiter, validate({ body: loginSchema }), login);
router.post("/refresh", refresh);
router.post("/logout", logout);

module.exports = router;
```

Events & announcements (pagination, search, RSVP)

Aim. Admin-managed events with student RSVP; admin announcements; all reads authenticated.

```
const mongoose = require("mongoose");

const EventSchema = new mongoose.Schema(
  {
    title: { type: String, required: true, trim: true },
    description: { type: String, default: "" },
    date: { type: Date, required: true },
    venue: { type: String, default: "" },
    createdBy: { type: mongoose.Schema.Types.ObjectId, ref: "User" },
    attendees: [{ type: mongoose.Schema.Types.ObjectId, ref: "User" }],
  },
  { timestamps: true }
);

module.exports = mongoose.models.Event || mongoose.model("Event", EventSchema);
```

```
const Event = require("../models/Event");

async function listEvents(req, res) {
  const page = Math.max(1, parseInt(req.query.page || "1", 10));
  const limit = Math.min(50, Math.max(1, parseInt(req.query.limit || "10", 10)));
  const search = (req.query.search || "").trim();
  const filter = search ? { title: { $regex: search, $options: "i" } } : {};
  const [items, total] = await Promise.all([
    Event.find(filter).sort({ date: 1 }).skip((page - 1) * limit).limit(limit).lean(),
    Event.countDocuments(filter),
  ]);
  return res.status(200).json({
    items,
    page,
    limit,
    total,
    totalPages: Math.max(1, Math.ceil(total / limit)),
  });
}

async function createEvent(req, res) {
  const { title, description, date, venue } = req.body;
  const event = await Event.create({
    title,
    description: description || "",
    date,
    venue: venue || "",
    createdBy: req.user.sub,
    attendees: [],
  });
  return res.status(201).json(event);
}

async function rsvpEvent(req, res) {
  const event = await Event.findById(req.params.id);
  if (!event) {
    return res.status(404).json({ error: "Event not found" });
  }
  const uid = req.user.sub;
  const ids = (event.attendees || []).map(String);
  if (!ids.includes(String(uid))) {
    event.attendees = [...(event.attendees || []), uid];
    await event.save();
  }
  return res.status(200).json(event);
}
```

```
module.exports = { listEvents, createEvent, rsvpEvent };
```

```
const express = require("express");
const { z } = require("zod");
const validate = require("../middleware/validate");
const { authenticate, authorize } = require("../middleware/auth");
const { listEvents, createEvent, rsvpEvent } = require("../controllers/eventController")
;

const router = express.Router();

const eventSchema = z.object({
  title: z.string().min(2).max(200),
  description: z.string().max(2000).optional(),
  date: z.string().min(1, "Date is required"),
  venue: z.string().max(200).optional(),
});

router.get("/", authenticate, listEvents);
router.post("/", authenticate, authorize("ADMIN"), validate({ body: eventSchema }), crea
teEvent);
router.post("/:id/rsvp", authenticate, authorize("ADMIN", "STUDENT"), rsvpEvent);

module.exports = router;
```

```
const Announcement = require("../models/Announcement");

async function listAnnouncements(req, res) {
  const items = await Announcement.find().sort({ createdAt: -1 }).limit(50).lean();
  return res.status(200).json(items);
}

async function createAnnouncement(req, res) {
  const { title, body } = req.body;
  const item = await Announcement.create({
    title,
    body: body || "",
    createdBy: req.user.sub,
  });
  return res.status(201).json(item);
}

module.exports = { listAnnouncements, createAnnouncement };
```

Tasks 5–6 — Hardening (helmet, CORS, rate-limit, zod) and app wiring

Aim. Secure defaults and structured 400s; single app factory also used by the tests.

```
const rateLimit = require("express-rate-limit");

// Max 5 login attempts per 15 minutes per IP (Task 5).
const loginLimiter = rateLimit({
  windowMs: 15 * 60 * 1000,
  max: 5,
  standardHeaders: true,
  legacyHeaders: false,
  message: { error: "Too many login attempts. Please try again after 15 minutes." },
});

module.exports = { loginLimiter };
```

```
// Generic zod validator: validate({ body: schema, query: schema, params: schema })
```

```
function validate(schemas) {
  return (req, res, next) => {
    try {
      if (schemas.body) req.body = schemas.body.parse(req.body);
      if (schemas.query) req.query = schemas.query.parse(req.query);
      if (schemas.params) req.params = schemas.params.parse(req.params);
      return next();
    } catch (err) {
      return res.status(400).json({
        error: "Validation failed",
        details: err.errors
          ? err.errors.map((e) => ({ path: e.path.join("."), message: e.message }))
          : err.message,
      });
    }
  };
}

module.exports = validate;
```

```
const express = require("express");
const helmet = require("helmet");
const cors = require("cors");
const cookieParser = require("cookie-parser");
const authRoutes = require("./routes/authRoutes");
const eventRoutes = require("./routes/eventRoutes");
const announcementRoutes = require("./routes/announcementRoutes");

function createApp() {
  const app = express();
  app.use(helmet());
  app.use(
    cors({
      origin: process.env.FRONTEND_ORIGIN || "http://localhost:5173",
      credentials: true,
    })
  );
  app.use(express.json());
  app.use(cookieParser());
  app.get("/api/health", (req, res) => res.json({ status: "ok" }));
  app.use("/api/auth", authRoutes);
  app.use("/api/events", eventRoutes);
  app.use("/api/announcements", announcementRoutes);
  // Central error handler — keeps stack traces out of API responses.
  // eslint-disable-next-line no-unused-vars
  app.use((err, req, res, next) => {
    console.error(err);
    res.status(err.status || 500).json({ error: err.message || "Internal server error" }
);
  });
  return app;
}

module.exports = createApp;
```

Task 6 (backend) — Jest + Supertest over in-memory mocks

Aim. Nine tests: register/token, weak-password 400, login ok/wrong-password, 401 without token, 403 student-on-admin-route, admin create, pagination+search, student RSVP.

Theory. Models are jest.mocked to in-memory stores so the suite runs with no MongoDB; real bcrypt/jwt/zod code paths execute.

```
const request = require("supertest");
```

```
jest.mock("../src/models/User", () => require("./mocks/inMemoryUser"));
jest.mock("../src/models/Event", () => require("./mocks/inMemoryEvent"));
jest.mock("../src/models/Announcement", () => require("./mocks/inMemoryAnnouncement"));

const createApp = require("../src/app");
const User = require("../src/models/User");

const app = createApp();

beforeEach(() => {
  User.__reset();
});

describe("POST /api/auth/register", () => {
  it("registers a new user and returns an access token", async () => {
    const res = await request(app).post("/api/auth/register").send({
      name: "Alice Student",
      email: "alice@campus.edu",
      password: "password123",
      role: "STUDENT",
    });
    expect(res.status).toBe(201);
    expect(res.body).toHaveProperty("accessToken");
    expect(res.body.user.email).toBe("alice@campus.edu");
    expect(res.body.user.role).toBe("STUDENT");
    expect(res.body.user.passwordHash).toBeUndefined();
  });

  it("rejects a weak password via zod validation (400)", async () => {
    const res = await request(app).post("/api/auth/register").send({
      name: "Bob",
      email: "bob@campus.edu",
      password: "123",
    });
    expect(res.status).toBe(400);
  });
});

describe("POST /api/auth/login", () => {
  beforeEach(async () => {
    await request(app).post("/api/auth/register").send({
      name: "Carol Admin",
      email: "carol@campus.edu",
      password: "supersecret1",
      role: "ADMIN",
    });
  });

  it("logs in with correct credentials", async () => {
    const res = await request(app).post("/api/auth/login").send({
      email: "carol@campus.edu",
      password: "supersecret1",
    });
    expect(res.status).toBe(200);
    expect(res.body).toHaveProperty("accessToken");
  });

  it("fails login with wrong password (401)", async () => {
... [31 lines truncated for print]
```

```
const request = require("supertest");

jest.mock("../src/models/User", () => require("./mocks/inMemoryUser"));
jest.mock("../src/models/Event", () => require("./mocks/inMemoryEvent"));
jest.mock("../src/models/Announcement", () => require("./mocks/inMemoryAnnouncement"));
```

```
const createApp = require("../src/app");
const User = require("../src/models/User");
const Event = require("../src/models/Event");

const app = createApp();

async function registerAdmin() {
  const res = await request(app).post("/api/auth/register").send({
    name: "Admin User",
    email: "admin@campus.edu",
    password: "adminpass123",
    role: "ADMIN",
  });
  return res.body.accessToken;
}

beforeEach(() => {
  User.__reset();
  Event.__reset();
});

describe("POST /api/events", () => {
  it("lets an ADMIN create an event successfully (201)", async () => {
    const token = await registerAdmin();
    const res = await request(app)
      .post("/api/events")
      .set("Authorization", `Bearer ${token}`)
      .send({
        title: "Annual Tech Fest",
        description: "Coding & robotics",
        date: "2026-11-05",
        venue: "Main Auditorium",
      });
    expect(res.status).toBe(201);
    expect(res.body.title).toBe("Annual Tech Fest");
    expect(res.body.createdBy).toBeDefined();
  });
});

describe("GET /api/events", () => {
  it("supports pagination and search-by-title", async () => {
    const token = await registerAdmin();
    await request(app).post("/api/events").set("Authorization", `Bearer ${token}`).send(
{
      title: "Tech Fest",
      date: "2026-11-05",
    });
    await request(app).post("/api/events").set("Authorization", `Bearer ${token}`).send(
{
      title: "Cultural Night",
      date: "2026-11-10",
    });
    await request(app).post("/api/events").set("Authorization", `Bearer ${token}`).send(
{
      title: "Tech Talk Series",
      date: "2026-11-15",
    });
... [28 lines truncated for print]
```

```
$ npx jest --runInBand
PASS tests/events.test.js
PASS tests/auth.test.js
Test Suites: 2 passed, 2 total
Tests: 9 passed, 9 total
```

Conclusion. Genuine 9/9 green. Live server start was not possible: no MongoDB in this environment

(server.js exits without it) — documented, not claimed. — Saiyed Shizain

Task 4 (core) — Minimal login/events frontend

Aim. In-memory JWT login plus search/pagination/RSVP list; full Redux shell arrives in Lab 10.

```
import axios from "axios";

// Lab Sheet 9 — Saiyed Shizain: token held in module memory (never localStorage).
let accessToken = null;
export function setAccessToken(token) {
  accessToken = token;
}

const api = axios.create({
  baseURL: import.meta.env.VITE_API_URL || "http://localhost:5000/api",
  withCredentials: true,
});

api.interceptors.request.use((config) => {
  if (accessToken) {
    config.headers.Authorization = `Bearer ${accessToken}`;
  }
  return config;
});

export default api;
```

```
import { useState } from "react";
import api, { setAccessToken } from "./api.js";

// Minimal login: stores the JWT in memory and hands the token up to App.
export default function Login({ onLogin }) {
  const [email, setEmail] = useState("");
  const [password, setPassword] = useState("");
  const [error, setError] = useState("");

  async function handleSubmit(e) {
    e.preventDefault();
    setError("");
    try {
      const { data } = await api.post("/auth/login", { email, password });
      setAccessToken(data.accessToken);
      onLogin(data.user, data.accessToken);
    } catch (err) {
      setError(err.response?.data?.error || "Login failed");
    }
  }

  return (
    <div className="card">
      <h2>CampusConnect Login</h2>
      <form onSubmit={handleSubmit}>
        <label htmlFor="email">Email</label>
        <input
          id="email"
          value={email}
          onChange={(e) => setEmail(e.target.value)}
          placeholder="you@campus.edu"
        />
        <label htmlFor="password">Password</label>
        <input
          id="password"
          type="password"
          value={password}
```

```
          onChange={(e) => setPassword(e.target.value)}
        />
        <button type="submit">Login</button>
      </form>
      {error && <p role="alert" className="error">{error}</p>}
    </div>
  );
}
```

```
import { useEffect, useState } from "react";
import api from "./api.js";

// Event list with search + pagination + RSVP against the Lab 9 API.
export default function EventList({ token }) {
  const [items, setItems] = useState([]);
  const [page, setPage] = useState(1);
  const [totalPages, setTotalPages] = useState(1);
  const [search, setSearch] = useState("");

  async function load(p = page, s = search) {
    const { data } = await api.get("/events", { params: { page: p, limit: 10, search: s
} });
    setItems(data.items);
    setPage(data.page);
    setTotalPages(data.totalPages);
  }

  useEffect(() => {
    load(1, search);
    // eslint-disable-next-line react-hooks/exhaustive-deps
  }, [search, token]);

  async function rsvp(id) {
    await api.post(`/events/${id}/rsvp`);
    load();
  }

  return (
    <div>
      <h2>Campus Events</h2>
      <input
        aria-label="search-events"
        placeholder="Search by title..."
        value={search}
        onChange={(e) => setSearch(e.target.value)}
      />
      <ul data-testid="event-list">
        {items.map((event) => (
          <li key={event._id}>
            <strong>{event.title}</strong> — {new Date(event.date).toLocaleDateString()}
            <button onClick={() => rsvp(event._id)}>RSVP</button>
          </li>
        ))}
      </ul>
      <div>
        <button disabled={page <= 1} onClick={() => load(page - 1)}>Prev</button>
        <span> Page {page} of {totalPages} </span>
        <button disabled={page >= totalPages} onClick={() => load(page + 1)}>Next</butto
n>
      </div>
    </div>
  );
}
```

```
$ npm run build → ✓ built
```

```
$ curl served index: 200
<title>CampusConnect — Saiyed Shizain</title>
```